

Answers to the final exam on Algebraic Specification

Christian Rinderknecht

17 June 2005

1 Binary tree specification

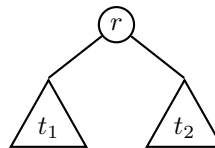
Let us recall an algebraic specification of binary trees over nodes. Let us call it $\text{BIN-TREE}(\text{node})$. Here is the signature:

- **Defined types**

- The type of the binary trees is noted $\mathbf{t}$.
- The type of the nodes is noted node .

- **Constructors**

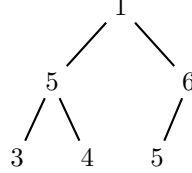
- $\text{EMPTY} : \mathbf{t}$
The term EMPTY represents the empty tree, i.e. the tree that contains no node.
- $\text{MAKE} : \text{node} \times \mathbf{t} \times \mathbf{t} \rightarrow \mathbf{t}$
The term $\text{MAKE}(r, t_1, t_2)$ denotes the tree whose root is r , left subtree is t_1 and right subtree is t_2 . Graphically:



- **Functions**

1. $\text{MEM} : \mathbf{t} \times \text{node} \rightarrow \text{bool}$
The term $\text{MEM}(t, e)$ is TRUE if node e occurs in tree t , otherwise it is equal to FALSE . (We assume we have equality on nodes.)
2. $\text{MIN-DEPTH} : \mathbf{t} \times \text{node} \rightarrow \text{int}$
The term $\text{MIN-DEPTH}(t, e)$ denotes the minimum depth at which node e occurs in tree t . (The root is at depth 0.) If e is not in t , the value is unspecified. In case the node e appears several times

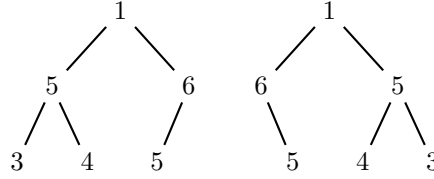
in t , the value is the smallest depth of occurrence. (We assume we can compare nodes for equality and that we have the function $\text{MIN} : \text{int} \times \text{int} \rightarrow \text{int}$ which returns the smallest integer argument.) For example, if t is



$$\text{then } \begin{cases} \text{MIN-DEPTH}(t, 7) & \text{is unspecified (i.e. undefined)} \\ \text{MIN-DEPTH}(t, 1) & = 0 \\ \text{MIN-DEPTH}(t, 3) & = 2 \\ \text{MIN-DEPTH}(t, 5) & = \text{MIN}(1, 2) = 1 \end{cases}$$

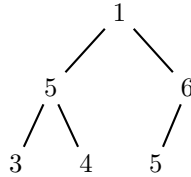
3. $\text{INV} : \mathbf{t} \rightarrow \mathbf{t}$

The term $\text{INV}(t)$ denotes the tree t in a mirror, i.e. the left subtrees of t are the right subtrees of $\text{INV}(t)$ and the right subtrees become the left subtrees. Therefore $\text{INV}(\text{INV}(t)) = t$. For example, the following trees are mirrors of each other:



4. $\text{SUM} : \text{BIN-TREE}(\text{int}).\mathbf{t} \rightarrow \text{int}$

The term $\text{SUM}(t)$ denotes the sum of all (integer) nodes in tree t . If t is empty, the sum is unspecified. For example, if t is



$$\text{then } \text{SUM}(t) = 1 + 5 + 3 + 4 + 6 + 5 = 24.$$

Question 1. Complete this signature with equations defining the functions.

Answer 1.

1. $\text{MEM} : \mathbf{t} \times \text{node} \rightarrow \text{bool}$

This function takes an argument of type $\mathbf{t}$, i.e. a binary tree. Since

binary trees can be constructed in two different ways, the first step is to write the left-hand sides with these constructors:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= ? \\ \text{MEM}(\text{MAKE}(r, t_1, t_2), e) &= ?\end{aligned}$$

The first equation is easy to complete: by definition, there is no node in the empty tree. Therefore

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{MAKE}(r, t_1, t_2), e) &= ?\end{aligned}$$

In the last equation, we notice that there are two nodes, r and e , so we can compare them for equality and split the equation correspondingly:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{MAKE}(e, t_1, t_2), e) &= ? \\ \text{MEM}(\text{MAKE}(r, t_1, t_2), e) &= ? \quad \text{if } r \neq e\end{aligned}$$

We can now complete the second equation because we know, by definition of the case, that e is in t (it is its root):

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{MAKE}(e, t_1, t_2), e) &= \text{TRUE} \\ \text{MEM}(\text{MAKE}(r, t_1, t_2), e) &= ? \quad \text{if } r \neq e\end{aligned}$$

In the case of the last equation, we know that e is not the root of t . Therefore, we must check whether e is not at some other place in t by means of recursive calls in the left and right subtrees of t :

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{MAKE}(e, t_1, t_2), e) &= \text{TRUE} \\ \text{MEM}(\text{MAKE}(r, t_1, t_2), e) &= \text{MEM}(t_1, e) \parallel \text{MEM}(t_2, e) \quad \text{if } r \neq e\end{aligned}$$

Actually, we can simplify the equations by merging back the third and second equations:

$$\begin{aligned}\text{MEM}(\text{EMPTY}, e) &= \text{FALSE} \\ \text{MEM}(\text{MAKE}(r, t_1, t_2), e) &= (r = e) \parallel \text{MEM}(t_1, e) \parallel \text{MEM}(t_2, e)\end{aligned}$$

2. MIN-DEPTH : $\mathbf{t} \times \mathbf{node} \rightarrow \mathbf{int}$

The first step is to note that MIN-DEPTH takes an argument of type $\mathbf{t}$, i.e. a binary tree, hence we can consider the two constructors for tree as this argument:

$$\begin{aligned}\text{MIN-DEPTH}(\text{EMPTY}, e) &= ? \\ \text{MIN-DEPTH}(\text{MAKE}(r, t_1, t_2), e) &= ?\end{aligned}$$

What is the minimum depth of any node e in the empty tree? Since, by definition, there is no node in the empty tree, there is no depth for any node: it is unspecified. Therefore, the first equation is meaningless, and we can remove it. Now remains

$$\text{MIN-DEPTH}(\text{MAKE}(r, t_1, t_2), e) = ?$$

There are two nodes, r and e , in the left-hand side of this equation, so we can compare them for equality and split the equation accordingly:

$$\begin{aligned} \text{MIN-DEPTH}(\text{MAKE}(e, t_1, t_2), e) &= ? \\ \text{MIN-DEPTH}(\text{MAKE}(r, t_1, t_2), e) &= ? \quad \text{if } r \neq e \end{aligned}$$

In the first equation, the node we are looking for is the root of t . Since the text explaining the signature tells that depth of the root is always 0, we just found the right-hand side:

$$\begin{aligned} \text{MIN-DEPTH}(\text{MAKE}(e, t_1, t_2), e) &= 0 \\ \text{MIN-DEPTH}(\text{MAKE}(r, t_1, t_2), e) &= ? \quad \text{if } r \neq e \end{aligned}$$

In the remaining equation, the node we search, e , is not the root of t . Thus we must search the left and right subtrees recursively and take the minimum of the two depth **plus one**:

$$\begin{aligned} \text{MIN-DEPTH}(\text{MAKE}(e, t_1, t_2), e) &= 0 \\ \text{MIN-DEPTH}(\text{MAKE}(r, t_1, t_2), e) &= 1 + \text{MIN}(\text{MIN-DEPTH}(t_1, e), \\ &\quad \text{MIN-DEPTH}(t_2, e)) \\ &\quad \text{if } r \neq e \end{aligned}$$

But something is wrong with the last equation... What if e is not in t_1 or not in t_2 ? Then the values $\text{MIN-DEPTH}(t_1, e)$ or $\text{MIN-DEPTH}(t_2, e)$ would be undefined, but what is the minimum of two undefined values? It is preferable then to constrain e to be in t in order to get a specified value $\text{MIN-DEPTH}(t, e)$:

$$\begin{aligned} \text{MIN-DEPTH}(\text{MAKE}(e, t_1, t_2), e) &= 0 \\ \text{MIN-DEPTH}(\text{MAKE}(r, t_1, t_2), e) &= 1 + \text{MIN}(\text{MIN-DEPTH}(t_1, e), \\ &\quad \text{MIN-DEPTH}(t_2, e)) \\ &\quad \text{if } r \neq e \text{ and } \text{MEM}(\text{MAKE}(r, t_1, t_2), e) \end{aligned}$$

3. $\text{INV} : \mathbf{t} \rightarrow \mathbf{t}$

The sole argument of INV is of type $\mathbf{t}$, i.e. it is a binary tree, so we start by enumerating the tree constructors:

$$\begin{aligned} \text{INV}(\text{EMPTY}) &= ? \\ \text{INV}(\text{MAKE}(r, t_1, t_2)) &= ? \end{aligned}$$

The first equation specifies the image of the empty tree in a mirror (inverse). Since, by definition, there is no node in the empty tree, its image can only be itself:

$$\begin{aligned}\text{INV}(\text{EMPTY}) &= \text{EMPTY} \\ \text{INV}(\text{MAKE}(r, t_1, t_2)) &= ?\end{aligned}$$

Now what about the non-empty tree case? First, we should understand that the invariant part is the root: the image of the root is at the same place:

$$\begin{aligned}\text{INV}(\text{EMPTY}) &= \text{EMPTY} \\ \text{INV}(\text{MAKE}(r, t_1, t_2)) &= \text{MAKE}(r, ?, ?)\end{aligned}$$

The subtrees have to permute, i.e. we swap them in order to get the reverse order **and** we reverse them also with a recursive call:

$$\begin{aligned}\text{INV}(\text{EMPTY}) &= \text{EMPTY} \\ \text{INV}(\text{MAKE}(r, t_1, t_2)) &= \text{MAKE}(r, \text{INV}(t_2), \text{INV}(t_1))\end{aligned}$$

4. SUM : BIN-TREE(int).t → int

We note that SUM as a unique argument of type BIN-TREE(int).t, i.e. it is a binary tree over positive integers. So we start by enumerating the tree constructors for this argument:

$$\begin{aligned}\text{SUM}(\text{EMPTY}) &= ? \\ \text{SUM}(\text{MAKE}(n, t_1, t_2)) &= ?\end{aligned}$$

The first equation refers to the sum of the nodes of the empty tree. Since, by definition, there is no node in the empty tree, the sum is therefore undefined. Hence we should remove this equation, but... what if the empty tree is part of a non-empty tree (like a tree just made of a root)? For instance, $\text{SUM}(\text{MAKE}(5, \text{EMPTY}, \text{EMPTY})) = 5$. In this case we expect the sum of the empty tree to be 0, because it should not modify the whole sum, so 5 actually comes from $5 + 0 + 0$. Thus we have to stick with

$$\begin{aligned}\text{SUM}(\text{EMPTY}) &= 0 \\ \text{SUM}(\text{MAKE}(n, t_1, t_2)) &= ?\end{aligned}$$

We already know that we have to sum n to the rest of the nodes:

$$\begin{aligned}\text{SUM}(\text{EMPTY}) &= 0 \\ \text{SUM}(\text{MAKE}(n, t_1, t_2)) &= n + ?\end{aligned}$$

We have to add now the sum of the left subtree and the sum of the right subtree nodes, as recursive calls:

$$\begin{aligned}\text{SUM}(\text{EMPTY}) &= 0 \\ \text{SUM}(\text{MAKE}(n, t_1, t_2)) &= n + \text{SUM}(t_1) + \text{SUM}(t_2)\end{aligned}$$

Question 2. Why a breadth-first search is better than a depth-first search in defining MIN-DEPTH?

Answer 2. If the tree is not well-balanced, i.e., if not all the leaves lay at the same level, then a breadth-first search would stop as soon as a leaf is found, whereas a deep-first search has to visit all the nodes.